声明：该 report 只对问题陈述和算法思路进行简单的说明，不粘贴代码实现和运行结果，因为这部分内容已经在 script 中给出。

1. Flowchart

问题陈述：

按照流程图创建一个函数 Print_values，变量为 a, b, c，给出流程图对应的输出并测试。

算法思路：

根据流程图分析得到函数目的是对 a, b, c 三个数按照从大到小的顺序进行重排。需要注意的点是流程图在 a<b 且 b>c 的情况下不考虑 a 和 c 的大小关系，也没有设计输出。因此，只需要按照流程图的顺序嵌套条件语句即可，对于上述的特殊情况使用 pass，函数就不会输出任何结果。

2. Matrix multiplication

问题陈述：

- 创建两个矩阵，M1(5*10), M2(10*5)，两个矩阵的所有元素都是 0 到 50 的随机整数。
- 只使用 for 循环，*算符和+算符写一个能实现矩阵乘法的函数 Matrix_multip。

算法思路：

- 使用 numpy 库中的 np.random.randint 即可实现。
- 使用三重 for 循环实现，循环 i 遍历 M1 的行，循环 j 遍历 M2 的列，循环 k 遍历 M1 的列（M2 的行）。通过 p[i, j] += M1[i, k] * M2[k, j]把 M1 第 i 行的元素和 M2 第 j 列的元素一一对应相乘累加，放置在结果矩阵的对应位置。另外，在函数中补充了要求 M1 列数和 M2 行数需要相等以确保它们的维度匹配。

3. Pascal triangle

问题陈述：

经典的帕斯卡三角（杨辉三角）问题，需要写一个函数 Pascal_triangle，输入行数能输出该行的完整数组。

算法思路：

杨辉三角恰好对应二项式展开的系数，所以这里直接利用的数学的公式。对于第 k 行，对应$(1+x)^k$展开的所有系数，即$C_k^i = \frac{i!(k-i)!}{k!}, i = 1,2,3,\ldots,k$。用一次 for 循环结合 factorial 函数即可实现。

4. Add or double

问题陈述：

从初始资金 1 元（1RMB）出发，每次操作可以选择将当前资金翻倍（乘以 2）或加 1 元。给定一个 1 到 100 之间的随机整数 x，编写一个函数 Least_moves 来计算并打印达到恰好 x 元所需的最小操作次数。

算法思路：

反向思考这个问题，现在有一个目标金额 x，如何让其最快到 1：如果是偶数，除以 2 是最优的；如果是大于 1 的奇数，那么减 1 变成偶数再除以 2，以此类推。因此使用 while 循环结合判断奇偶的条件语句，重复上述操作直至 x 的值为 1。其中为偶数时，只除以 2，操作次数+1；为奇数时，先减 1 再除以 2，因此操作次数+2。

5. Dynamic programming

问题陈述：

在数字序列 123456789 之间插入任意个 +、-保持数字原有顺序不变，构成数学表达式。

- 编写函数 Find_expression，给定一个 1 到 100 之间的随机整数 x，打印出所有计算结果等于 x 的表达式。
- 统计从 1 到 100 每个整数所对应的表达式总数，将数量依次存入列表并绘图，找到表达式数量最多和最少对应的 x 值。

算法思路：

- 如果只考虑+、-符号，那这个问题很难思考，因为我们并不知道具体要插入几个、插入在什么位置。因此，我们考虑到空字符，在没有插入加减号的位置可以被视为插入了一个空字符，这样，在“123456789”这个字符串中，它的 8 个间隔每个都有 3 种选择，即“+”, “-”, “”。因此一共有 3^8 中排列方式。那么用最笨的方法，用 8 个 for 循环遍历所有组合，并在最后把它和数字拼接即可得到表达式的字符串。当然，如何把这个字符串变成代码可计算的也很重要，这里借用了 eval 函数，它可以很好地实现这一目的。例如 eval(“1+2”)会输出 3。
- 为了实现计数可行达式数量的目的，在 Find_expression 函数中加入了简单的一行 n += 1，另外设置 p 为 Find_expression 的一个新自变量，起到控制输出的作用，如果 p 取 1，则输出所有满足条件的解的表达式；如果 p 为其他值，则只输出一共有多少种解数 n。防止 Total_solutions 出现多余的量。画图通过 matplotlib.pyplot 实现。找到表达式数量最多和最少对应的 x 值简单应用 for 循环和列表索引即可实现。